CS3014 - Estructuras de Datos Avanzadas

Strings II
Semana 13

Víctor Racsó Galván Oyola

vgalvan@utec.edu.pe

3 de noviembre de 2026

Objetivos de la sesión



Objetivos de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Definir el suffix array y buscar patrones con él
- Construirlo en $O(n \lg n)$ con duplicación de prefijos

Objetivos de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Definir el suffix array y buscar patrones con él
- Construirlo en $O(n \lg n)$ con duplicación de prefijos
- Calcular el arreglo LCP con Kasai
- Resolver los problemas clásicos de subcadenas y conocer cómo se construye en $O(n)$ (DC3)

Motivación

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Ahora el texto se conoce primero

KMP y Aho–Corasick preprocesan los **patrones**. Si el **texto** T es fijo y llegan muchos patrones P distintos, cada búsqueda en $O(|T|)$ es demasiado.

Motivación

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Ahora el texto se conoce primero

KMP y Aho–Corasick preprocesan los **patrones**. Si el **texto** T es fijo y llegan muchos patrones P distintos, cada búsqueda en $O(|T|)$ es demasiado.

Indexar el texto

Todo patrón que ocurre en T es un **prefijo de algún sufijo** de T. Si los sufijos están **ordenados**, los que empiezan con P forman un intervalo contiguo.

Suffix array

Convención y definición

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Sentinela

Todo texto termina con \$, menor que cualquier carácter y único. Así ningún sufijo es prefijo de otro y la comparación es siempre decidible.

Suffix array

Para una cadena S de longitud n (con \$), SA es la **permutación** de $0, \ldots, n-1$ tal que

$$S[SA[0]..] \prec S[SA[1]..] \prec \cdots \prec S[SA[n-1]..]$$

Solo n enteros: mucho menos memoria que un árbol.

Ejemplo: $S = \texttt{banana\$}$

k	$SA[k]$	sufijo
0	6	$
1	5	a$
2	3	ana$
3	1	anana$
4	0	banana$
5	4	na$
6	2	nana$

Los sufijos que empiezan con ana son $k = 2, 3$: un intervalo. Hay 2 ocurrencias, en las posiciones 3 y 1.

Búsqueda con búsqueda binaria

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Buscar P

Dos búsquedas binarias sobre SA comparando P con el sufijo $S[SA[k]..]$: el menor k con sufijo $\succeq P$ y el menor con sufijo que **no** empieza con P y es $\succ P$. Entre ambos están todas las ocurrencias.

Búsqueda con búsqueda binaria

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Buscar P

Dos búsquedas binarias sobre SA comparando P con el sufijo $S[SA[k]..]$: el menor k con sufijo $\succeq P$ y el menor con sufijo que **no** empieza con P y es $\succ P$. Entre ambos están todas las ocurrencias.

Costo

Cada comparación cuesta $O(|P|)$ y hay $O(\lg n)$: $O(|P| \lg n)$. Contar ocurrencias es la longitud del intervalo; listarlas cuesta $O(\text{ocurrencias})$ más.

Con información de LCP se baja a $O(|P| + \lg n)$.

Duplicación de prefijos

¿Cómo construirlo?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Ordenar los sufijos tal cual

Es una ordenación con comparador. Cada comparación de dos sufijos cuesta hasta $O(n)$: $O(n^2 \lg n)$ en total. Para $n = 10^6$ es inviable.

Lo que cuesta es comparar

Si pudiéramos comparar dos sufijos en $O(1)$, serían $O(n \lg n)$. ¿Cómo comparar en $O(1)$ sin mirar caracteres?

La idea de Manber y Myers

Ordenar por los primeros 2^k caracteres

Sea $rank_k(i)$ la posición de $S[i..i+2^k-1]$ cuando se ordenan todas las subcadenas de longitud 2^k (con empates iguales). Entonces

$$S[i..i+2^{k+1}-1] \prec S[j..j+2^{k+1}-1]$$
$$\iff (rank_k(i), rank_k(i+2^k)) \prec (rank_k(j), rank_k(j+2^k))$$

Cada nivel sabe comparar en $O(1)$

Una subcadena de longitud 2^{k+1} son dos de longitud 2^k. Basta comparar el **par de rangos**. Tras $\lceil \lg n \rceil$ niveles, las subcadenas de longitud $\geq n$ son sufijos completos.

Ejemplo: $S = \texttt{banana\$}$

i	0	1	2	3	4	5	6
$S[i]$	b	a	n	a	n	a	\$
$rank_0$ (1 carácter)	2	1	3	1	3	1	0

Ejemplo: $S = \texttt{banana\$}$

i	0	1	2	3	4	5	6
$S[i]$	b	a	n	a	n	a	\$
$rank_0$ (1 carácter)	2	1	3	1	3	1	0
$rank_1$ (2 caracteres)	3	2	4	2	4	1	0
$rank_2$ (4 caracteres)	4	3	6	2	5	1	0

Los rangos de $i = 1$ y $i = 3$ coinciden hasta longitud 2 (an) y se separan en longitud 4 (anan frente a ana\$). Cuando todos son distintos, *rank* es la permutación inversa de SA.

Una ordenación por conteo por nivel

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Clave de dos componentes: radix sort

Ordenar por la segunda componente y luego, de forma **estable**, por la primera.

La segunda componente sale gratis

Si a ya está ordenado por los primeros 2^k caracteres, entonces ordenar por $rank_k(i + 2^k)$ equivale a recorrer a y tomar $a[t] - 2^k$ (módulo n): el índice $a[t] - 2^k$ tiene como segunda componente la clave de $a[t]$.

Luego un **counting sort** estable por la primera componente: $O(n)$ por nivel.

Manber y Myers

Algoritmo 1: SUFFIX-ARRAY(S) *(S termina en \$; $n = |S|$)*

$a \leftarrow$ índices ordenados por $S[i]$; $rank \leftarrow$ rangos por $S[i]$; $len \leftarrow 1$
mientras *len* $< n$ **y** *los rangos no son todos distintos* **hacer**
  para $t \leftarrow 0$ **a** $n-1$ **hacer** $sec[t] \leftarrow (a[t] - len + n)$ mód n
  ordenar *sec* de forma estable por $rank[\cdot]$ con *counting sort* $\rightarrow a$
  $nuevo[a[0]] \leftarrow 0$
  para $t \leftarrow 1$ **a** $n-1$ **hacer**
    $x \leftarrow (rank[a[t]],\ rank[(a[t] + len) \text{ mód } n])$
    $y \leftarrow (rank[a[t-1]],\ rank[(a[t-1] + len) \text{ mód } n])$
    $nuevo[a[t]] \leftarrow nuevo[a[t-1]] + [\,x \neq y\,]$
  $rank \leftarrow nuevo$; $len \leftarrow 2 \cdot len$
devolver a

$O(n \lg n)$ tiempo, $O(n)$ memoria.

Resumen de la construcción

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Complejidad

Variante	Tiempo	Memoria
Ordenar sufijos con `std::sort`	$O(n^2 \lg n)$	$O(n)$
Duplicación con ordenación	$O(n \lg^2 n)$	$O(n)$
Duplicación con counting sort	$O(n \lg n)$	$O(n)$

Más adelante

Hay algoritmos de tiempo $O(n)$ (DC3, que veremos al final, y SA-IS, que solo mencionaremos). Para casi todo problema de competencia, $O(n \lg n)$ basta.

El arreglo LCP

LCP de sufijos adyacentes

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Definición

$LCP[k] = \text{LCP}\big(S[SA[k-1]..],\ S[SA[k]..]\big)$ para $k = 1, \ldots, n-1$: el prefijo común de dos sufijos **vecinos** en el orden.

Para banana\$

k	0	1	2	3	4	5	6
$SA[k]$	6	5	3	1	0	4	2
$LCP[k]$	–	0	1	3	0	0	2

¿Por qué solo los vecinos?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

LCP de dos sufijos cualquiera

Para $k < l$:

$$\text{LCP}\big(S[SA[k]..], S[SA[l]..]\big) = \text{mín}\{LCP[k+1], \ldots, LCP[l]\}$$

Demostración

Los sufijos entre ellos están ordenados, así que todos comparten el prefijo común de los extremos: cada uno comparte con el siguiente **al menos** el LCP de los extremos, y en algún par adyacente se pierde justo ese carácter.

Cómo calcular LCP: Kasai

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Observación

Sea $S[j..]$ el sufijo que va **inmediatamente antes** de $S[i..]$ en el orden, con $\text{LCP} = h > 0$. Quitar el primer carácter (común) conserva el orden y deja $h-1$ caracteres comunes: $S[j+1..] \prec S[i+1..]$ y comparten $h-1$.

Cómo calcular LCP: Kasai

Observación

Sea $S[j..]$ el sufijo que va **inmediatamente antes** de $S[i..]$ en el orden, con $\text{LCP} = h > 0$. Quitar el primer carácter (común) conserva el orden y deja $h-1$ caracteres comunes: $S[j+1..] \prec S[i+1..]$ y comparten $h-1$.

Consecuencia

El predecesor de $S[i+1..]$ está entre $S[j+1..]$ y $S[i+1..]$ (o es $S[j+1..]$), así que comparte con $S[i+1..]$ **al menos** $h-1$ caracteres. Si procesamos las posiciones $i = 0, 1, 2, \ldots$ y reutilizamos h, solo baja en 1 de una a la siguiente.

Algoritmo de Kasai

Algoritmo 2: KASAI(S, SA)

para $k \leftarrow 0$ **a** $n-1$ **hacer** $rank[SA[k]] \leftarrow k$
$h \leftarrow 0$
para $i \leftarrow 0$ **a** $n-1$ **hacer**
  si $rank[i] > 0$ **entonces**
    $j \leftarrow SA[rank[i]-1]$
    mientras $i+h < n$ **y** $j+h < n$ **y** $S[i+h] = S[j+h]$ **hacer** $h \leftarrow h+1$
    $LCP[rank[i]] \leftarrow h$
    si $h > 0$ **entonces** $h \leftarrow h-1$
  en otro caso
    $h \leftarrow 0$
devolver LCP

$O(n)$

h baja a lo más 1 por iteración y nunca es mayor que n: el **while** avanza $\leq 2n$ veces en total.

Aplicaciones

Subcadenas distintas

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Contar las subcadenas distintas de S

Toda subcadena es prefijo de algún sufijo; el sufijo $SA[k]$ aporta los prefijos que **no** compartía con el anterior:

$$\#\text{distintas} = \frac{n(n+1)}{2} - \sum_k LCP[k]$$

con $n = |S|$ **sin** contar el sentinela.

Para banana

$\frac{6 \cdot 7}{2} = 21$ prefijos de sufijos, menos $\sum LCP = 0 + 1 + 3 + 0 + 0 + 2 = 6$: 15 subcadenas distintas.

Más aplicaciones

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Subcadena repetida más larga

Es el $\max_k LCP[k]$ (dos sufijos que comparten ese prefijo).

Subcadena común de dos cadenas

Concatenar $S \# T$ y calcular SA y LCP. La subcadena común más larga es el mayor $LCP[k]$ entre dos sufijos vecinos que **vienen de cadenas distintas**.

La k-ésima subcadena

Con LCP, la subcadena número k en orden lexicográfico (sin repetidas) se halla recorriendo SA y restando $n - SA[k] - LCP[k]$ nuevos prefijos por sufijo.

Ventajas y límites

Ventajas	n enteros de memoria; fácil de programar; con LCP resuelve casi todo lo de subcadenas
Límites	búsqueda $O(\|P\| \lg n)$ sin ayudas; construir en $O(n)$ es complejo; el **suffix tree** resuelve algunas consultas con más naturalidad

Construcción en tiempo lineal

La estrategia común

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Ordenar poco y deducir mucho

Los algoritmos lineales siguen el mismo esquema:

1. Elegir un subconjunto **pequeño** de sufijos y ordenarlo **recursivamente** sobre una cadena más corta.
2. **Deducir** el orden de todos los demás a partir de ese subconjunto, en tiempo lineal.

Recurrencia

Si la cadena de la recursión tiene $\leq \alpha n$ caracteres con $\alpha < 1$ y el resto cuesta $O(n)$:

$$T(n) = T(\alpha n) + O(n) = O(n)$$

Algoritmos históricos

Autores	Año	Tiempo	Idea
Manber y Myers	1990	$O(n \lg n)$	duplicación
Itoh y Tanaka	1999	superlineal	ordenamiento inducido
Kärkkäinen y Sanders (DC3)	2003	$O(n)$	mod 3
Ko y Aluru	2003	$O(n)$	ordenamiento inducido
Nong, Zhang y Chan (SA-IS)	2009	$O(n)$	inducido, simplificado

Hoy: DC3 (el más fácil de demostrar); SA-IS (el más usado) solo se menciona.

Partir las posiciones por su residuo

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Tres clases

$$B_k = \{\, i : \ 0 \leq i < n, \ i \equiv k \pmod 3 \,\}, \qquad k = 0, 1, 2$$

Sea $C = B_1 \cup B_2$: dos tercios de las posiciones.

Teorema (Kärkkäinen, Sanders y Burkhardt)

Si se conoce el orden de los sufijos de C, se puede deducir el orden de todos en $O(n)$.

Primero hay que ordenar C: aquí entra la recursión.

Paso 1: ordenar los sufijos de C

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Tomar *tripletas*

Para cada $i \in C$ se toma la tripleta $S[i]S[i+1]S[i+2]$ (rellenando con \$ si se sale). Ordenarlas con radix sort (3 pasadas) y **nombrarlas** con su rango.

La cadena reducida

$R = R_1 \cdot R_2$: los nombres de las tripletas de B_1 en orden de posición, y luego los de B_2. Tiene $\approx \frac{2n}{3}$ caracteres. Un sufijo de C en S es una **tripleta seguida de otros sufijos de tripletas**: queda igual al sufijo de R.

Ejemplo: $S = \texttt{mississippi}$

$$R_1 = (\texttt{iss})(\texttt{iss})(\texttt{ipp})(\texttt{i\$\$}), \quad R_2 = (\texttt{ssi})(\texttt{ssi})(\texttt{ppi})$$

Nombres

$\texttt{i\$\$} \rightarrow 0, \quad \texttt{ipp} \rightarrow 1, \quad \texttt{iss} \rightarrow 2, \quad \texttt{ppi} \rightarrow 3, \quad \texttt{ssi} \rightarrow 4.$

$$R = 2\,2\,1\,0\,4\,4\,3$$

Hay nombres repetidos (2 y 4): se llama recursivamente a DC3 sobre R. Si todos fueran distintos, el rango de cada tripleta daría directamente el orden.

El resultado de la recursión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Rangos de los sufijos de C

El suffix array de $R = 2210443$ da, para cada posición de R, su rango entre los sufijos de C:

posición en R	0	1	2	3	4	5	6
R	2	2	1	0	4	4	3
rnk	3	2	1	0	6	5	4

De vuelta a S

La posición $0, 1, 2, 3$ de R son las posiciones $1, 4, 7, 10$ de S (B_1) y $4, 5, 6$ son $2, 5, 8$ (B_2). Así el sufijo en 10 tiene rango 0, el de 7 rango 1, etc.

Paso 3: mezclar

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Dos listas ordenadas

Los sufijos de C (paso 1) y los de B_0 (paso 2) se mezclan como en MergeSort. Falta decidir cómo comparar $x \in B_0$ con $y \in C$ en $O(1)$.

Las dos comparaciones

- $x \in B_0$, $y \in B_1$: comparar $\big(S[x],\ rnk(x+1)\big)$ con $\big(S[y],\ rnk(y+1)\big)$ ($x+1 \in B_1$, $y+1 \in B_2$: ambos con rango).
- $x \in B_0$, $y \in B_2$: comparar $\big(S[x], S[x+1],\ rnk(x+2)\big)$ con $\big(S[y], S[y+1],\ rnk(y+2)\big)$ ($x+2 \in B_2$, $y+2 \in B_1$).

DC3: algoritmo

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Algoritmo 3: $\mathrm{DC3}(S, n)$

$B_k \leftarrow \{i : i \equiv k \text{ mód } 3\}$; $\quad C \leftarrow B_1 \cup B_2$
ordenar C por la tripleta $(S[i], S[i+1], S[i+2])$ con radix sort
$R \leftarrow$ nombres de las tripletas (B_1 y luego B_2)
si *los nombres no son todos distintos* **entonces**
  $SA_R \leftarrow \mathrm{DC3}(R, |R|)$
en otro caso
  $SA_R \leftarrow$ orden directo por nombre
$rnk[x] \leftarrow$ posición de $x \in C$ en SA_R
ordenar B_0 por el par $(S[i], rnk[i+1])$ con radix sort
devolver mezcla de B_0 y C con las comparaciones de $O(1)$

DC3: algoritmo

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Algoritmo 4: $\mathrm{DC3}(S, n)$

$B_k \leftarrow \{i : i \equiv k \text{ mód } 3\}$; $\quad C \leftarrow B_1 \cup B_2$
ordenar C por la tripleta $(S[i], S[i+1], S[i+2])$ con radix sort
$R \leftarrow$ nombres de las tripletas (B_1 y luego B_2)
si *los nombres no son todos distintos* **entonces**
  $SA_R \leftarrow \mathrm{DC3}(R, |R|)$
en otro caso
  $SA_R \leftarrow$ orden directo por nombre
$rnk[x] \leftarrow$ posición de $x \in C$ en SA_R
ordenar B_0 por el par $(S[i], rnk[i+1])$ con radix sort
devolver mezcla de B_0 y C con las comparaciones de $O(1)$

$$T(n) = T(\tfrac{2n}{3}) + O(n) = O(n)$$

Todos los demás pasos son radix sorts y una mezcla. La recursión es sobre $\frac{2}{3}n$ caracteres.

SA-IS: el más rápido en la práctica

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Una construcción lineal más

SA-IS (Nong, Zhang y Chan, 2009) construye el suffix array en $O(n)$ con una constante muy baja: es el que usan las bibliotecas y el más rápido en la práctica.

Por qué no lo vemos hoy

Su teoría es muy densa para una sola clase: clasifica los sufijos en tipo S y L, ordena los LMS, **induce** el orden del resto y reduce recursivamente. Quien lo necesite puede usar una implementación de biblioteca, sabiendo que produce el mismo *SA*.

Comparación y elección

	Tiempo	Recursión	Constante	Uso
Duplicación	$n \lg n$	no	baja	concursos, hasta $\sim 10^6$
DC3	n	$\frac{2}{3}n$	alta	didáctico, demostración
SA-IS	n	$\frac{1}{2}n$	baja	bibliotecas, $n \geq 10^6$

Después, el LCP

El LCP se calcula con Kasai en $O(n)$ para cualquiera de los tres. Con SA-IS + Kasai + RMQ, todo el índice se construye en tiempo lineal.

Práctica

Práctica

Problema	**Idea**
Library Checker — Suffix Array	implementar duplicación o DC3
Library Checker — Number of Substrings	SA + LCP
CSES 2104 — Pattern Positions	intervalo en SA + posiciones
CSES 2105 — Distinct Substrings	$\frac{n(n+1)}{2} - \sum LCP$
CSES 2106 — Repeating Substring	máximo de LCP
CSES 2108 — Substring Order I	k-ésima subcadena distinta
CSES 2109 — Substring Order II	k-ésima subcadena con repetidas
CSES 2110 — Substring Distribution	conteo por longitud con LCP
CSES 1113 — String Transform	transformada de Burrows–Wheeler
CSES 3225 — Inverse Suffix Array	reconstruir la cadena desde SA

Cierre de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- El suffix array ordena los sufijos; las ocurrencias de un patrón son un intervalo y se buscan con búsqueda binaria.

Cierre de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- El suffix array ordena los sufijos; las ocurrencias de un patrón son un intervalo y se buscan con búsqueda binaria.
- La duplicación de prefijos compara subcadenas de longitud 2^{k+1} con un par de rangos: $O(n \lg n)$ con counting sort.
- El arreglo LCP se calcula en $O(n)$ con Kasai; el LCP de dos sufijos es el mínimo de LCP entre sus rangos.

Cierre de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- El suffix array ordena los sufijos; las ocurrencias de un patrón son un intervalo y se buscan con búsqueda binaria.
- La duplicación de prefijos compara subcadenas de longitud 2^{k+1} con un par de rangos: $O(n \lg n)$ con counting sort.
- El arreglo LCP se calcula en $O(n)$ con Kasai; el LCP de dos sufijos es el mínimo de LCP entre sus rangos.
- Tiempo lineal: DC3 ordena dos tercios por tripletas y mezcla; SA-IS es aún más rápido, pero su teoría es muy densa para esta clase.

Logros de la sesión



Logros de la sesión

- Sabemos construir un suffix array por duplicación
- Sabemos calcular el LCP con Kasai

Logros de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Sabemos construir un suffix array por duplicación
- Sabemos calcular el LCP con Kasai
- Sabemos buscar, contar y resolver problemas de subcadenas
- Sabemos explicar cómo se logra el tiempo lineal con DC3

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Gracias

Referencias

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Bibliografía citada

- Manber, U., & Myers, G. (1993). Suffix arrays: a new method for on-line string searches. *SIAM J. Comput.*, 22(5), 935–948.
- Kasai, T., et al. (2001). Linear-time longest-common-prefix computation in suffix arrays and its applications. *CPM'01*, 181–192.
- Kärkkäinen, J., Sanders, P., & Burkhardt, S. (2006). Linear work suffix array construction. *J. ACM*, 53(6), 918–936.
- Nong, G., Zhang, S., & Chan, W. H. (2009). Linear suffix array construction by almost pure induced-sorting. *DCC'09*, 193–202.

Agradecimientos

Estas diapositivas están basadas en las notas de clase de Strings del curso (Manber–Myers, DC3, Ko–Aluru y SA-IS) y en las clases de Luciano A. Romero Calla para este curso.
Lista de créditos: Luciano A. Romero Calla, Erik Demaine, Keith Schwarz.